

Laboratorio 9

Información

Integrantes:

Name	Institution ID	GitHub User
Josué Say	22801	JosueSay
Carlos Valladares	221164	vgcarlol

- [Repositorio](#)

Preparación de entorno

```
In [ ]: # %pip install -r requirements.txt
# jupyter nbconvert Lab9.ipynb --to html
```

Usted ha sido contratado como Ingeniero de Visión por Computadora (CV Engineer) principal para la convención de anime y tecnología más grande de su ciudad: el *Akihabara Fest*. La organización requiere automatizar la logística y seguridad del evento utilizando cámaras de circuito cerrado (CCTV) y dispositivos Edge (Raspberry Pi 5 y NVIDIA Jetson Nano). Tendrá que tomar decisiones arquitectónicas críticas, resolver problemas de oclusión en multitudes de cosplayers y escribir un *pipeline* de inferencia desde cero.

Task 1 - Investigación

Además de las cámaras de seguridad con YOLO, los organizadores del *Akihabara Fest* quieren instalar "Cabinas Fotográficas de Realidad Aumentada (AR)" para los asistentes VIP. El objetivo de estas cabinas es doble:

1. Eliminar automáticamente el fondo real de la convención y reemplazarlo por escenarios de anime (ej. *La Aldea Oculta de la Hoja de Naruto* o *Neo-Tokyo de Akira*), simulando un "Chroma Key" (pantalla verde) perfecto sin usar tela verde.
2. Identificar específicamente la espada o báculo mágico que sostiene el usuario para superponerle un efecto de "brillo de energía" digital, diferenciándolo de las armas de otros amigos que estén en la misma foto.

La mesa directiva no sabe qué modelo de IA usar y le pide a usted investigar dos arquitecturas famosas: **U-Net** y **Mask R-CNN**.

Entonces, realice una investigación en artículos o documentación técnica oficial y redacte un reporte ejecutivo (máximo 2 páginas) respondiendo con rigor técnico a lo siguiente:

Inciso 1

Diferencia Fundamental: Defina con sus propias palabras la diferencia exacta a nivel de píxel entre la Segmentación Semántica y la Segmentación de Instancia.

Respuesta:

La **segmentación semántica** responde: *¿a qué categoría pertenece este píxel?* Cada píxel recibe exactamente una etiqueta de clase (persona, fondo, espada, escenario), pero la tarea no distingue entre individuos de la misma categoría. Si hay tres personas en la imagen, los píxeles de las tres quedan etiquetados como "persona" con el mismo color, fundidos en una única región sin fronteras entre ellas. Es una clasificación densa de píxeles, no de objetos.

La **segmentación de instancia** responde: *¿a qué objeto específico pertenece este píxel?* Aquí cada objeto detectado recibe su propia máscara binaria independiente, diferenciando no solo la categoría sino la identidad. Si hay tres personas, cada una tiene su propia máscara: Persona-1, Persona-2, Persona-3. Dos píxeles del mismo color de piel pueden pertenecer a instancias distintas y el modelo lo sabe.

Inciso 2

El Caso U-Net: Explique por qué la arquitectura U-Net (Segmentación Semántica) es ideal y altamente eficiente para la tarea 1 (separar a todos los humanos del fondo), pero explique técnicamente por qué fracasaría si intentamos usarla para la tarea 2 si hay dos réplicas de espadas idénticas cruzadas en la foto.

Respuesta:

Por qué U-Net es ideal para la Tarea 1:

U-Net es una arquitectura encoder-decoder diseñada para producir una máscara de segmentación del mismo tamaño que la imagen de entrada. Su encoder contrae la imagen progresivamente extrayendo características semánticas (¿qué hay aquí?), y su decoder las expande recuperando la resolución espacial para asignar una etiqueta a cada píxel con precisión de contorno. Las **skip connections** entre capas simétricas del encoder y el decoder son la clave: transfieren información de alta resolución que el decoder usa para reconstruir bordes nítidos sin perder detalle espacial.

Para separar a todos los humanos del fondo del escenario de convención, la tarea es exactamente lo que U-Net resuelve: etiquetar todos los píxeles que pertenecen a la clase "persona" como primer plano y el resto como fondo. El modelo no necesita saber cuántas personas hay ni dónde termina una y empieza otra; solo necesita responder "¿este píxel es humano o no?". U-Net lo hace de forma eficiente con una sola pasada en una arquitectura relativamente ligera.

Por qué fracasaría en la Tarea 2:

Si dos cosplayers cruzan sus espadas idénticas en la foto, U-Net ve la región combinada de ambas espadas y la etiqueta como "espada", un único bloque de píxeles sin distinción de pertenencia. Producirá una sola máscara fusionada para toda la región de espada visible.

Superponer efectos de brillo diferenciado (rojo para una espada, azul para otra) requiere saber exactamente qué píxeles pertenecen a la espada del Usuario-A y cuáles a la del Usuario-B. U-Net no puede responder esa pregunta porque su salida es un mapa de categorías, no un mapa de identidades.

Inciso 3

El Caso Mask R-CNN: Explique cómo la arquitectura Mask R-CNN (Segmentación de Instancia) resuelve el problema anterior basándose en su naturaleza de "dos etapas" (recordando que hereda de Faster R-CNN). ¿Por qué Mask R-CNN sí podría iluminar una espada de rojo y otra de azul, incluso si se están tocando en la imagen?

Respuesta:

Mask R-CNN hereda directamente de Faster R-CNN y añade una tercera cabeza de salida a su arquitectura de dos etapas:

1. El backbone (ResNet + FPN) procesa la imagen completa y genera un feature map compartido.
2. La RPN propone regiones candidatas sobre ese feature map, igual que en Faster R-CNN.
3. Para cada propuesta, **Roi Align** (versión mejorada de Roi Pooling que usa interpolación bilineal en lugar de cuantización discreta) extrae un tensor de características de tamaño fijo.
4. Ese tensor alimenta simultáneamente tres cabezas independientes: clasificación de clase, regresión de bounding box, y **predicción de máscara binaria** de $m \times m$ píxeles para esa región.

La clave de por qué puede iluminar una espada de rojo y otra de azul está en que cada instancia tiene su **propia máscara generada de forma independiente**, asociada a su propia bounding box y su propio ID de detección. El modelo no decide primero qué son todos los píxeles de la imagen; primero detecta y separa los objetos como instancias individuales (etapa de propuesta + clasificación), y luego genera la máscara para cada uno por separado dentro de su región acotada.

Si las dos espadas se están tocando en la imagen, la RPN genera dos bounding boxes distintas (una por espada), dos pasadas independientes por Roi Align y dos máscaras independientes. Aunque los píxeles de las espadas se solapan espacialmente en la imagen original, el sistema los trata como dos objetos distintos desde el momento en que la RPN los propone como regiones separadas. Cada máscara puede recibir un efecto de brillo distinto en postprocesamiento porque cada una tiene un identificador único de instancia.

Task 2

En el pabellón principal del evento, se está llevando a cabo un concurso de cosplay. De repente, un grupo de 15 personas haciendo cosplay de Naruto entran al escenario exactamente iguales y posan muy juntos (simulando el *Kage Bunshin no Jutsu* o Jutsu Clones de Sombra). Las cámaras registran a los clones abrazados, superpuestos y parcialmente ocluidos unos detrás de otros.

Usted nota que su modelo clásico basado en YOLOv8 falla rotundamente: la red neuronal sí encuentra a los 15 clones y dibuja cientos de cajas, pero al final en la pantalla solo aparecen 4 o 5 personas detectadas. El resto "desaparece". Considerando esto, responda:

Inciso 1

Explique matemáticamente, utilizando la fórmula del IoU (Intersección sobre Unión), por qué el algoritmo NMS está causando que los clones superpuestos desaparezcan (Falsos Negativos).

La fórmula es: $IoU = \frac{|I|}{|U|}$

Respuesta:

El algoritmo NMS elimina cajas con alto IoU respecto a la caja de mayor confianza seleccionada. El problema en la escena de los 15 clones de Naruto es que los cosplayers están físicamente superpuestos y abrazados, lo que significa que sus bounding boxes reales se solapan masivamente.

Consideremos dos clones adyacentes. Si el Clone-A ocupa la región $[x_1, y_1, x_2, y_2]$ y el Clone-B, parcialmente detrás, ocupa $[x'_1, y'_1, x'_2, y'_2]$, y ambas regiones se solapan en un área $|I|$ significativa, el IoU entre sus cajas puede ser fácilmente:

$$IoU = \frac{|I|}{|U|} = \frac{|I|}{|b_A| + |b_B| - |I|}$$

Si los clones están abrazados con un 60% de solapamiento, el IoU puede superar 0.5, que es el umbral por defecto de NMS. El algoritmo NMS ordena todas las cajas por confianza, selecciona la del Clone-A (mayor score), calcula el IoU con todas las demás, y elimina al Clone-B porque $IoU > \theta_{NMS}$. Desde la perspectiva de NMS, dos cajas con ese nivel de solapamiento son redundantes y corresponden al mismo objeto, así que descarta la de menor confianza.

Al ser dos personas distintas que comparten espacio físico en la imagen. NMS no tiene forma de saberlo porque toma decisiones puramente geométricas basadas en solapamiento, sin entender que la escena tiene objetos genuinamente separados cuyas proyecciones 2D se fusionan. El resultado es que los clones del fondo son sistemáticamente eliminados como si fueran detecciones duplicadas del mismo clon del frente, generando los falsos negativos observados.

Inciso 2

Si usted es el ingeniero a cargo y solo puede modificar los hiperparámetros en el código durante la inferencia: ¿Qué pasaría si ajusta el umbral de IoU del NMS a 0.15? ¿Qué pasaría si lo ajusta a 0.95?

Justifique qué valor sería más adecuado para este problema de alta densidad.

Respuesta:

Con $\theta_{NMS} = 0.15$:

NMS eliminaría cualquier caja que se solape más del 15% con la seleccionada. En la escena de los clones, incluso cosplayers que están uno al lado del otro sin tocarse físicamente tendrían un IoU mayor a 0.15 por la simple proximidad espacial. El sistema detectaría quizás 2 o 3 personas de las 15, eliminando a casi todos los clones reales como si fueran duplicados. Más falsos negativos que la configuración actual.

Con $\theta_{NMS} = 0.95$:

NMS solo elimina cajas que se solapan en un 95% o más, lo que significa que prácticamente solo descarta copias casi idénticas del mismo objeto. En la escena de los clones, la mayoría de las cajas redundantes del mismo clon sobrevivirían, pero también lo harían casi todas las cajas de clones distintos. El conteo de personas mejora, aunque a costa de tener múltiples detecciones por clon en algunos casos.

Valor recomendado para alta densidad:

Para este escenario concreto, un umbral en el rango de 0.6–0.7 sería el balance más adecuado. Es suficientemente alto para no eliminar clones distintos que se solapan parcialmente (como los que están abrazados con IoU de 0.4–0.5), pero suficientemente bajo para suprimir las cajas verdaderamente redundantes que apuntan exactamente al mismo clon desde posiciones muy similares (IoU > 0.7).

Inciso 3

Si el presupuesto le permitiera cambiar el modelo a YOLOv10, explique cómo su arquitectura de **Asignación Dual de Etiquetas (Dual Label Assignment)** resolvería este problema de oclusión sin necesidad de tocar el NMS.

Respuesta:

YOLOv10 resuelve el problema desde la raíz eliminando la necesidad de NMS en inferencia por completo, mediante la **Asignación Dual de Etiquetas (Dual Label Assignment)**.

Durante el entrenamiento, YOLOv10 opera con dos ramas de asignación paralelas:

- **Rama one-to-many:** cada objeto real puede ser asignado a múltiples predictores simultáneamente durante el entrenamiento. Esto genera una señal de supervisión rica y variada, permitiendo que la red aprenda representaciones robustas del objeto desde múltiples perspectivas de cuadrícula. Es la forma en que los detectores clásicos entrenan.

- **Rama one-to-one:** adicionalmente, para cada objeto real existe exactamente un predictor designado como el único responsable de detectarlo en inferencia. Esta rama entrena al modelo para que, al momento de inferir, cada objeto real tenga exactamente una predicción ganadora, sin duplicados.

La clave está en que la restricción "un predictor por objeto" no se aplica como postprocesamiento externo (como hace NMS), sino que está aprendida durante el entrenamiento. El modelo internaliza que debe suprimir sus propias predicciones redundantes, y la rama one-to-one lo obliga a seleccionar la predicción más informada para cada instancia.

Con los 15 clones, en YOLOv10 no existe un paso de NMS que compare geométricamente las cajas y elimine las que se solapan. El modelo ya fue entrenado para emitir exactamente una detección por persona, independientemente de cuánto se solapen sus bounding boxes en la imagen. Dos clones abrazados con IoU de 0.7 entre sus cajas no son un problema de postprocesamiento, porque el sistema nunca necesita calcular ese IoU en inferencia; cada clon ya tiene su predictor designado desde el entrenamiento, y ambos emiten su detección de forma autónoma sin interferir con el otro.

Task 3

El cliente quedó fascinado con sus propuestas teóricas y ahora quiere un Prototipo Funcional (MVP). Desean una especie de "Pokedex" en tiempo real usando la cámara web de una computadora portátil para escanear y clasificar objetos en la convención.

Usted debe escribir un *script* de Python desde cero utilizando la librería ultralytics y OpenCV. No es necesario entrenar un modelo personalizado; puede utilizar los pesos pre-entrenados estándar (COCO dataset) o descargar un modelo pre-entrenado de la comunidad (ej. detección de caras de anime o cartas TCG), pero el código de inferencia debe ser suyo.

El script que realice debe tener:

1. **Captura de Video:** El programa debe abrir la cámara web (o leer un video mp4 pregrabado de temática geek que usted proporcione) utilizando OpenCV (cv2.VideoCapture).
2. **Inferencia Continua:** Instanciar un modelo de la familia YOLO (usted elige la versión, pero debe comentarlo en el código). Procesar cada *frame* capturado.
3. **Manipulación de Hiperparámetros:** El código debe pasar explícitamente como argumentos los valores de `conf` (Confidence threshold) y `iou` (NMS IoU threshold) a la función de inferencia del modelo.
4. **Cálculo de FPS:** Usted debe calcular algorítmicamente y en tiempo real los FPS (Frames Per Second) de la inferencia utilizando la librería `time`, y sobreponer ese número en la esquina del video final.

5. **Extracción de Tensores:** Para cada detección, extraiga las coordenadas (x_1, y_1, x_2, y_2) del tensor de salida y la etiqueta (clase), y dibújelas usando funciones primitivas de OpenCV (`cv2.rectangle`, `cv2.putText`), **no** usando el método `.plot()` automático de la librería. Esto demuestra que usted sabe iterar sobre el tensor de resultados.